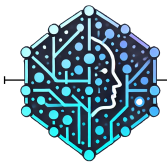


Laboratorio di Intelligenza Artificiale Applicata

a.k.a. AI Applications In Practice

7 - Matplotlib

Prof. Tiberio Uricchio
< tiberio.uricchio@unipi.it >

Vs code Jupyter Integration

Jupyter

Jupyter Notebooks are interactive computing environments that enable users to create and share documents containing live code, equations, visualizations, and narrative text.

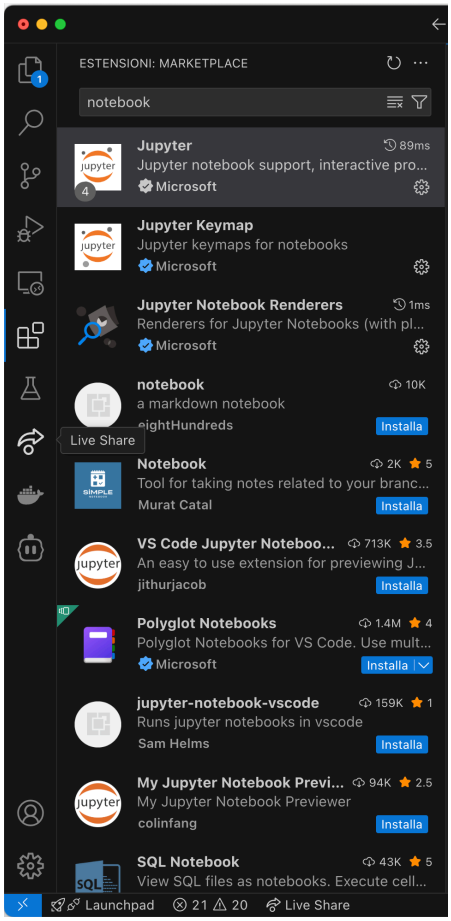
Key features:

- Combination of code and documentation in a single document
- Support for multiple programming languages (primarily Python)
- Interactive execution of code cells
- Rich output display (text, images, HTML, LaTeX equations)
- Web-based interface accessible through browsers

Vs Code and Jupyter

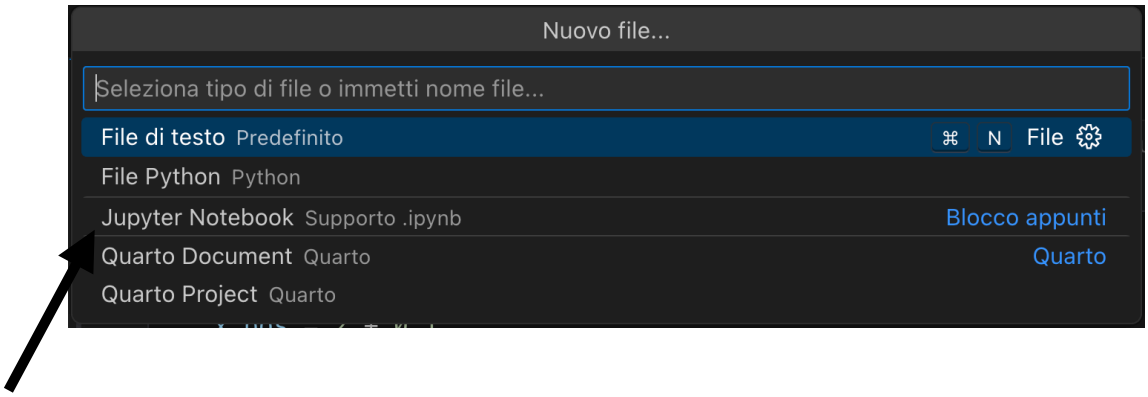
Jupyter is integrated into Vs Code with extensions that allows the creation of notebook directly in the interface.

To enable the support, add the *Jupyter*, *Jupyter Keymap* and the *Jupyter Notebook Renderers* extensions

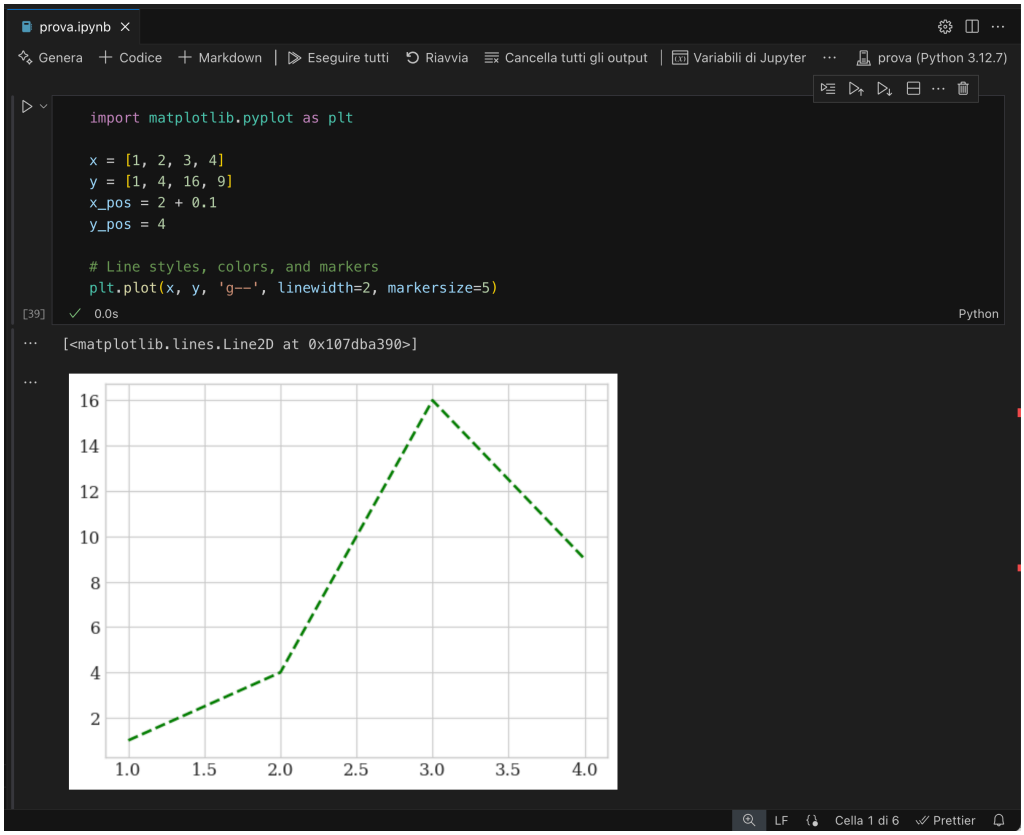


Vs Code and Jupyter

After installing the extensions you will be able to create notebooks directly from the new file selection dropdown



Vs Code and Jupyter

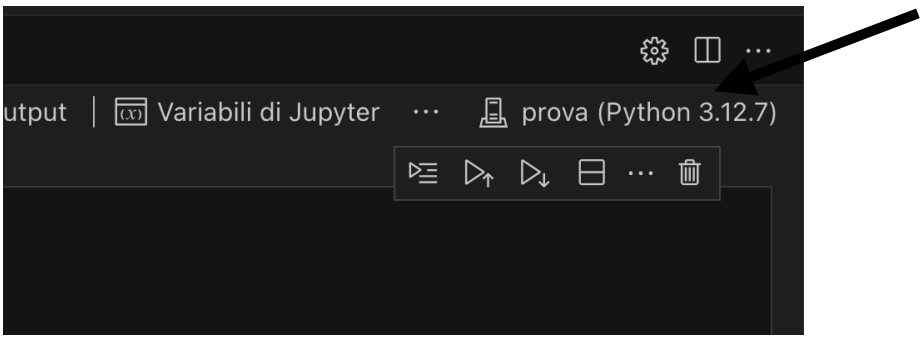


Vs Code and Jupyter

First thing, you have to choose the “kernel”, i.e. the version of python that you want to run, or a server Jupyter-compatible that is hosting a kernel.

Starting a kernel means starting a python interpreter that will remain active for the whole session.

Running a cell means sending the commands included in the cell to the active kernel.



Notebook Interface

The notebook interface consists of several key components:

- **Cells**: Basic units that contain either code or text (markdown)
- **Toolbar**: Contains buttons for common actions (save, add cell, run cell)
- **Kernel**: Backend engine that executes the code (e.g., Python 3)

Cell Types

Notebooks support different cell types for different purposes:

- **Code cells**: contain executable code; output appears below the cell
- **Markdown cells**: contain formatted text using Markdown syntax
- **Raw cells**: content is displayed without any processing

You can change a cell's type using the dropdown menu in the toolbar.

Code Execution

Code cells can be executed in multiple ways:

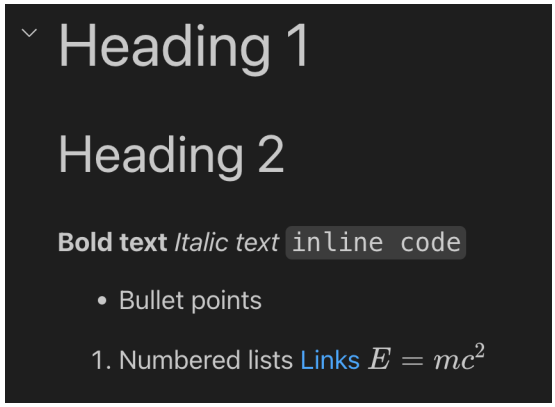
1. Click the "Run" button in the toolbar
2. Use keyboard shortcut Shift+Enter (run cell and move to next)
3. Use keyboard shortcut Ctrl+Enter (run cell and stay on current)
4. Use keyboard shortcut Alt+Enter (run cell and insert new below)

Code execution is handled by the kernel, and the output is displayed directly below the cell.

Markdown cells

Markdown cells allow you to document your code with rich text formatting:

```
# Heading 1
## Heading 2
**Bold text**
*Italic text*
`inline code`
- Bullet points
1. Numbered lists
[Links](https://www.example.com)
$E = mc^2$
```



Notably, it supports LaTeX formatting.

Markdown reference:

https://www.markdownguide.org/getting-started/

Magic commands

Special commands called "magic commands" can enhance functionality:

Cell magics (start with %%): Apply to the entire cell

- %%time: Time the execution of the entire cell
- %%HTML: Render the cell as HTML
- %%bash: Run the cell content as a bash script

Line magics (start with %): Apply to a single line

- %timeit: Measure execution time of a line
- %pwd: Print working directory
- %matplotlib inline: Configure matplotlib for inline plotting

Best Practices

Notebooks are mainly useful as a playground tool, to explore data and for testing functionality. Entire fixed flows are better maintained in standard python scripts.

The main risk is when running cells in out of order manner. It's easy to lose the correct flow and variables, since are global, may be changed in unintended ways.

For effective use of notebooks:

- Keep code cells small and focused on a single task
- Add markdown documentation between code sections
- Execute cells in order to ensure reproducibility
- Restart kernel and run all cells to verify notebook works correctly
- Use clear, descriptive variable names and add comments
- Save notebooks frequently and use version control

MATPLOTLIB

Matplotlib

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. It's designed to be simple yet powerful, making it the standard plotting library for Python programming.

Key features:

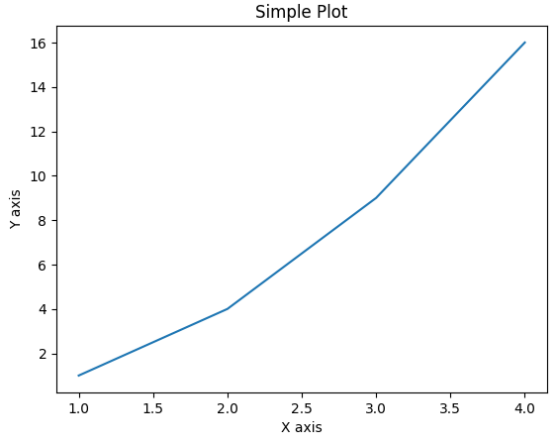
- Create publication-quality plots
- Make interactive figures that can zoom, pan, update
- Customize visual style and layout
- Export to many file formats (PNG, PDF, SVG, etc.)
- Integrate well with Numpy

Matplotlib

Matplotlib has two main interfaces:

- pyplot - A MATLAB-like state-based interface (most common)
- Object-oriented API - More powerful for complex figures

```python
import matplotlib.pyplot as plt

# Simple plot using pyplot
plt.plot([1, 2, 3, 4], [1, 4, 9, 16])
plt.title("Simple Plot")
plt.xlabel("X axis")
plt.ylabel("Y axis")
plt.show()
```



Matplotlib

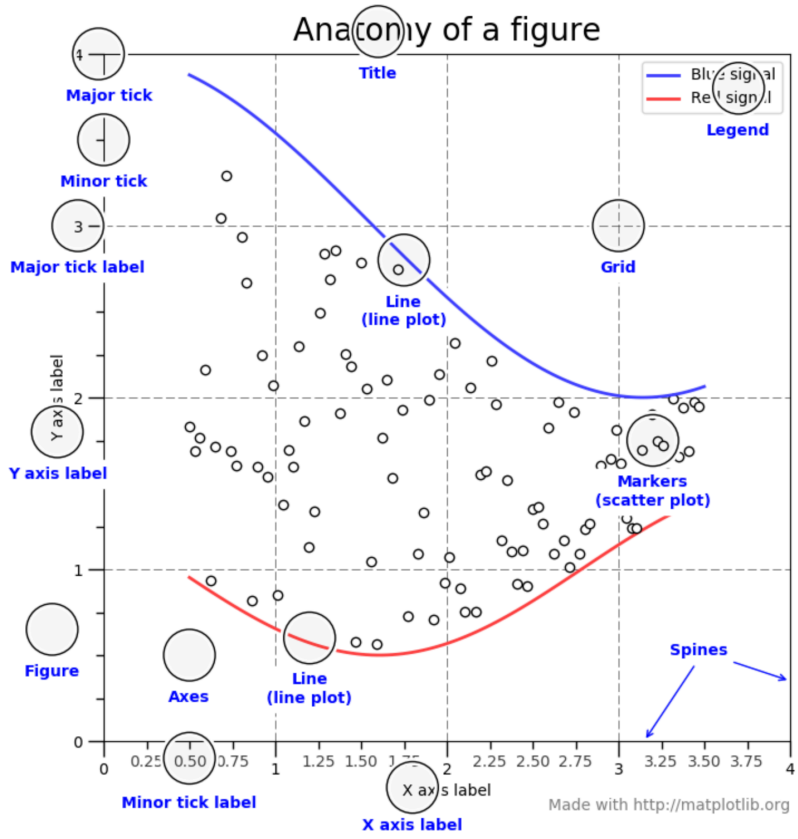
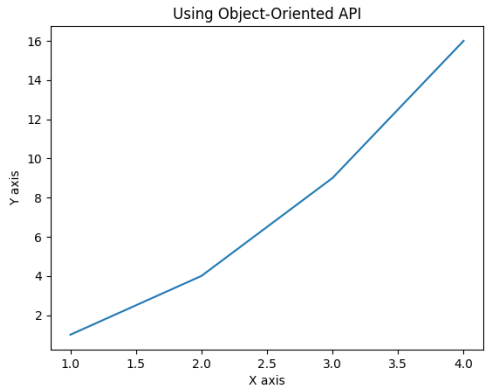


Figure and Axes

A Matplotlib visualization consists of:

- **Figure**: The overall window or page that contains everything
- **Axes**: An area where data is plotted with a particular coordinate system

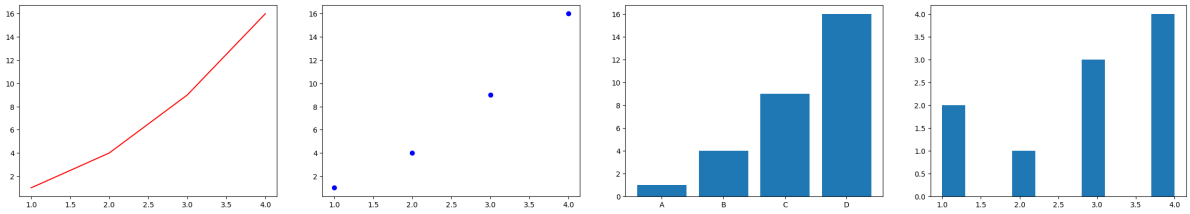
```
fig, ax = plt.subplots()  # Create a figure and an axes
ax.plot([1, 2, 3, 4], [1, 4, 9, 16])
ax.set_title("Using Object-Oriented API")
ax.set_xlabel("X axis")
ax.set_ylabel("Y axis")
plt.show()
```



Creating Basic Plots

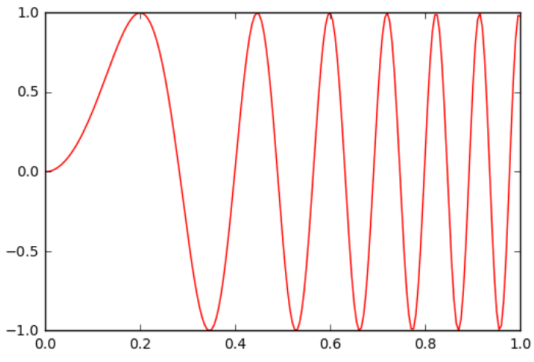
Matplotlib supports various plot types through simple function calls:

```
# Line plot
plt.plot([1, 2, 3, 4], [1, 4, 9, 16], 'r-')  # red line

# Scatter plot
plt.scatter([1, 2, 3, 4], [1, 4, 9, 16], c='blue', marker='o')

# Bar chart
plt.bar(['A', 'B', 'C', 'D'], [1, 4, 9, 16])

# Histogram
plt.hist([1, 1, 2, 3, 3, 3, 4, 4, 4, 4])
```



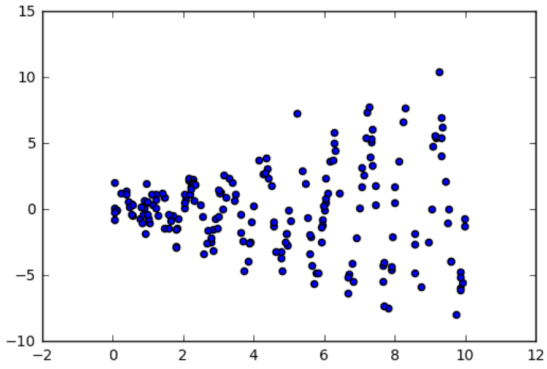
Matplotlib

```
In [1]: import numpy as np

In [2]: import matplotlib.pyplot as plt

In [3]: x = np.linspace(0, 1, 201)

In [4]: y = np.sin((2*np.pi*x)**2)

In [5]: plt.plot(x, y, 'red')

In [6]: plt.show()
```



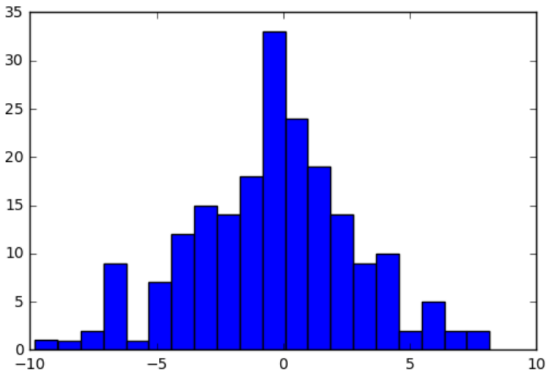
Matplotlib

```
In [1]: import numpy as np

In [2]: import matplotlib.pyplot as plt

In [3]: x = 10*np.random.rand(200,1)

In [4]: y = (0.2 + 0.8*x) * np.sin(2*np.pi*x) + \
   ...:     np.random.randn(200,1)

In [5]: plt.scatter(x,y)

In [6]: plt.show()
```



Matplotlib

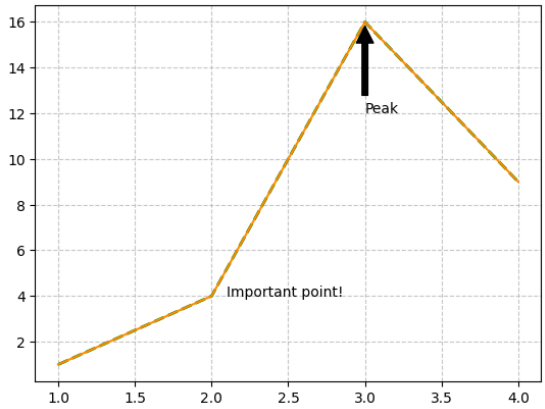
```
In [1]: import numpy as np

In [2]: import matplotlib.pyplot as plt

In [3]: x = 10*np.random.rand(200,1)

In [4]: y = (0.2 + 0.8*x) * \
   ...:     np.sin(2*np.pi*x) + \
   ...:     np.random.randn(200,1)

In [5]: plt.hist(y, bins=20)

In [6]: plt.show()
```



Customizing Plot Appearance

Matplotlib provides extensive options to customize plots:

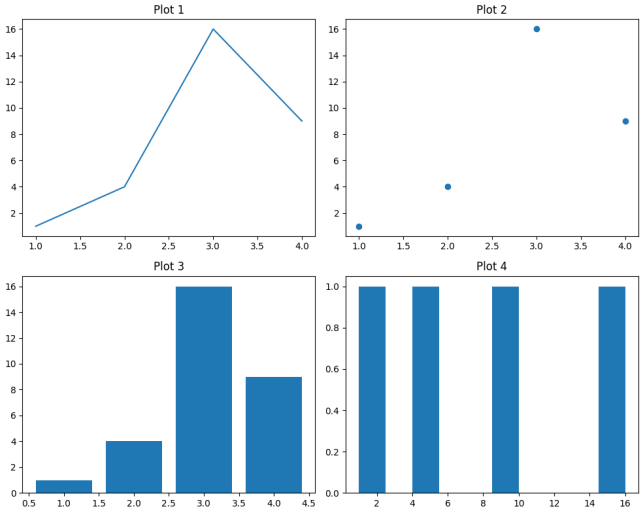
```
x = [1, 2, 3, 4]
y = [1, 4, 16, 9]
x_pos = 2 + 0.1
y_pos = 4

# Line styles, colors, and markers
plt.plot(x, y, 'g--', linewidth=2, markersize=5)

# Adding grid
plt.grid(True, linestyle='--', alpha=0.7)

# Custom colors
plt.plot(x, y, color='#FF8C00')  # Dark orange

# Text and annotations
plt.text(x_pos, y_pos, "Important point!")
plt.annotate("Peak", xy=(3, 16), xytext=(3, 12),
             arrowprops=dict(facecolor='black', shrink=0.05))
```



Multiple Subplots

Arrange multiple plots in a grid layout:

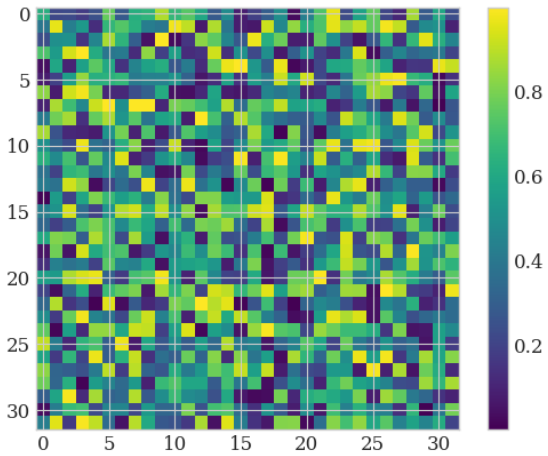
```
# 2x2 grid of subplots
fig, axs = plt.subplots(2, 2, figsize=(10, 8))

# Plot on each subplot
axs[0, 0].plot(x, y)
axs[0, 0].set_title('Plot 1')

axs[0, 1].scatter(x, y)
axs[0, 1].set_title('Plot 2')

axs[1, 0].bar(x, y)
axs[1, 0].set_title('Plot 3')

axs[1, 1].hist(y)
axs[1, 1].set_title('Plot 4')

# Adjust spacing between subplots
plt.tight_layout()
```



Advanced Customization

You can also use predefined styles and fine tune them

```
import numpy as np
data = np.random.rand(32,32)

# Custom styles
plt.style.use('seaborn-v0_8-whitegrid')

# Setting a global theme
plt.rcParams['font.family'] = 'serif'
plt.rcParams['font.size'] = 12
plt.rcParams['axes.grid'] = True

# Colormaps for complex visualizations
plt.imshow(data, cmap='viridis')
plt.colorbar()
```



Saving Figures

Save your visualizations in different formats:

```
# Save as PNG (raster image)
plt.savefig('my_plot.png', dpi=300)

# Save as PDF (vector image)
plt.savefig('my_plot.pdf')

# Save as SVG (vector image for web)
plt.savefig('my_plot.svg')

# Controlling figure size and resolution
plt.figure(figsize=(10, 6))  # Width, height in inches
plt.plot(x, y)
plt.savefig('high_quality.png', dpi=300, bbox_inches='tight')
```

Exercises

- Create a simple 2D line plot. Plot the *sin(x)* function between 0 to 10.
- Generate 50 random (x,y) points and plot them on a 2D scatter plot. Add a label for the x and y axes, a title for the plot and add a legend.
- Create a bar chart where each bar corresponds to a category in {A,B,C,D} and respective values are [5, 7, 3, 8]. Use a different color for each bar.
- Generate 1000 random points from a normal distribution (mu = 0, sigma = 1) and plot a histogram with 30 bins.